Constraint Programming (CP(M)) 2026

Sample Questions for the Invigilated Quiz

School of Computing Science, University of Glasgow

About these questions. The invigilated quiz on Wednesday 28 October consists of 25 single-answer multiple-choice questions, each worth 1 mark, covering the sessions up to and including 21 October: introduction to CP and combinatorial optimisation, search and arc consistency, global constraints, and other solvers. The five questions below are in the same style to some of the questions and are spread across those topics. They are *not* questions from the quiz.

Question 1.

Here is a correct MiniZinc model of the vertex cover problem as a decision problem. A *vertex cover* is a set of vertices that contains at least one endpoint of every edge.

```
int: n;                                % number of vertices
int: m;                                % number of edges
int: k;                                % cover size we are allowed
set of int: EDGE = 1..m;
array[EDGE] of 1..n: from;
array[EDGE] of 1..n: to;
array[1..n] of var 0..1: x;            % x[i] = 1 if vertex i is in the cover

constraint sum(i in 1..n)(x[i]) <= k;
constraint forall(e in EDGE)(
    x[from[e]] + x[to[e]] >= 1
);

solve satisfy;
```

The instance has $n = 5$, $m = 5$, $k = 3$, and edges 1–2, 2–3, 3–4, 4–5, 5–1 (a five-cycle).

Which of the following assignments to `x` does **not** satisfy the model?

A. `x = [1, 0, 1, 1, 0]`
B. `x = [1, 1, 0, 1, 0]`
C. `x = [1, 1, 1, 0, 0]`
D. `x = [0, 1, 0, 1, 1]`

Question 2.

Consider the CSP $X = \{p, q, r\}$, $D(p) = D(q) = D(r) = \{1, 2, 3\}$, with constraints $C_1 : p \neq q$, $C_2 : q \neq r$ and $C_3 : p < r$. We are part-way through the arc consistency algorithm from lecture. The algorithm has just processed the arc (r, C_3) and removed the value 1 from $D(r)$.

Which arcs must be added to the queue as a result?

A. (r, C_2) and (r, C_3)
B. (q, C_2) and (p, C_3)
C. (p, C_3) only
D. (p, C_1), (q, C_1), (q, C_2) and (p, C_3)

Question 3.

Consider the CSP $X = \{a, b, c\}$ with $D(a) = D(b) = D(c) = \{1, 2, 3, 4\}$ and constraints $a < b$ and $b < c$. What are the domains after arc consistency has been enforced, using the version of arc consistency we discussed in lecture?

A. $D(a) = \{1, 2\}$, $D(b) = \{2, 3\}$, $D(c) = \{3, 4\}$
B. $D(a) = \{1\}$, $D(b) = \{2\}$, $D(c) = \{3\}$
C. $D(a) = \{1, 2, 3\}$, $D(b) = \{2, 3\}$, $D(c) = \{3, 4\}$
D. $D(a) = \{1, 2\}$, $D(b) = \{2, 3\}$, $D(c) = \{4\}$

Question 4.

A `circuit` constraint is posted over `succ` for a 5-node problem, where `succ`$[i]$ is the node that follows node i. During search, `succ`$[2] = 4$ and then `succ`$[4] = 1$ have been fixed, so the fixed edges form the chain $2 \to 4 \to 1$.

What does the **sub-tour pruning rule** from lecture (the chain rule maintained with union-find) remove at this point?

A. Nothing — the rule only acts once the chain contains every node
B. The value 1 from $D($`succ`$[2])$, because node 1 is already on the chain
C. The value 4 from $D($`succ`$[1])$, because node 4 already has a predecessor
D. The value 2 from $D($`succ`$[1])$, because `succ`$[1] = 2$ would close a cycle of length $3 < 5$

Question 5.

During DPLL, the variable y appears in the remaining clauses only as the negative literal $\neg y$, never as y. What does the **pure literal** rule do?

A. Branches on y, trying *true* first and then *false*
B. Sets y to *false* and removes every clause containing $\neg y$
C. Sets y to *true* and removes $\neg y$ from every clause containing it
D. Nothing — the pure literal rule applies only to clauses containing a single literal

---

End of sample questions. Answers overleaf.

Answers

Question 1. **Answer: C.**

Question 2. **Answer: B.**

Question 3. **Answer: A.**

Question 4. **Answer: D.**

Question 5. **Answer: B.**